

Deep Learning — Week 3 Notes

Sigmoid Neurons · Loss & Supervised Setup · Gradient Descent · Feedforward Networks · Backprop by Chain Rule · Representation Power

NPTEL / IIT Ropar · CS7015 Lecture 3 · Exam-focused revision sheet

1 · Why Sigmoid? (From Harsh Step to Smooth Curve)

The perceptron's step is too harsh

A perceptron fires only when the weighted sum crosses the threshold ($-w_0$). This is a **step function**: a movie with criticsRating 0.51 is "like" but 0.49 is "dislike" — a sudden jump from 0 to 1. Real tasks need a **smooth**, gradual decision.

Perceptron (step): $y = 1$ if $\sum w_i x_i \geq -w_0 \rightarrow$ Sigmoid (smooth): $y = 1 / (1 + e^{-(w_0 + \sum w_i x_i)})$

- The sigmoid (logistic) neuron replaces the sharp jump with a smooth S-curve; no sudden transition at the threshold.
- Output is real-valued in **(0, 1)**; it is **continuous and differentiable** — this is what makes gradient-based learning possible.
- Now inputs x and outputs y can be **real numbers** (in $\mathbb{R}^n$), not just boolean $\{0,1\}$.

2 · The Supervised Learning Setup

Four ingredients

- **Data**: $\{(x_i, y_i)\}$ input–label pairs.
- **Model**: the function $\hat{y} = f(x; w)$ (e.g. sigmoid neuron).
- **Loss**: measures how wrong predictions are.
- **Learning algorithm**: adjusts parameters w to minimize loss (PLA, **gradient descent**, ...).

Squared-error loss

$$L(w) = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2$$

The learning algorithm's job is to find the **w that minimizes L** . Guessing parameters by hand is infeasible for many weights $\rightarrow$ we need a principled rule.

3 · Gradient Descent (Taylor-Series Justification)

Move in the direction that lowers the loss

We want a step $\Delta\theta$ that makes the new loss smaller: $L(\theta + \eta u) - L(\theta) < 0$. A first-order **Taylor expansion** shows the loss drops fastest when we move **opposite to the gradient**.

Update rule: $w \leftarrow w - \eta \cdot \partial L / \partial w$ (η = learning rate)

- The gradient $\partial L / \partial w$ points in the direction of **steepest increase**; we step the negative way to decrease L .
- The gradient shows the **local rate of change** of loss with respect to each parameter — not a class label, and it does not remove features.
- Repeat until convergence. Small η = slow but stable; large η = risk overshooting.

Worked update (Case Study 4 — recommendation score)

Linear model $\hat{y} = w_1x_1 + w_2x_2 + b$, with $x = (1, 2)$, $y = 6$, initial $w_1=w_2=1$, $b=0$, $\eta=0.1$, $L = \frac{1}{2}(\hat{y} - y)^2$.

Step	Computation	Result
Prediction $\hat{y}$	$1(1) + 1(2) + 0$	3
$\partial L / \partial \hat{y}$	$(\hat{y} - y) = 3 - 6$	-3
$\partial L / \partial w_2$	$(\hat{y} - y) \cdot x_2 = -3(2)$	-6
new w_1	$1 - 0.1 \cdot (-3 \cdot 1)$	1.3
new b	$0 - 0.1 \cdot (-3)$	0.3

Pattern to memorize: $\partial L / \partial w_j = (\hat{y} - y) \cdot x_j$ and $\partial L / \partial b = (\hat{y} - y)$. Each new parameter = old - $\eta \cdot$ (its gradient). A too-high prediction (negative error) **increases** the weights.

4 · Feedforward Neural Networks (The Forward Pass)

Information flows one way: inputs → hidden → output

A feedforward net computes **hidden quantities first**, then combines them into the output. Nothing is fed back; the target label is never used during the forward pass.

- **ORDER** Hidden units must be computed **before** the output $\hat{y}$ can be evaluated.
- **FLOW** "Feedforward" = information flows from inputs, through hidden quantities, to output.
- Hidden layers with a nonlinearity let the network represent transformations a single unit cannot.

Case Study 1 trace — $z_1 = 2x_1 + x_2$, $z_2 = x_1 - x_2 + 1$, $\hat{y} = z_1 + 2z_2$, with $x = (2, 1)$:

$z_1 = 2(2)+1 = \mathbf{5}$ · $z_2 = 2-1+1 = \mathbf{2}$ · $\hat{y} = 5 + 2(2) = \mathbf{9}$.

5 · Backpropagation via the Chain Rule (Computational Graphs)

Push the error backward through intermediate quantities

Backprop computes gradients from the loss **back through** the output and hidden computations, using the **chain rule**. Draw a computational graph, do the forward pass, then multiply local derivatives backward.

$$\partial L / \partial x = (\partial L / \partial v) \cdot (\partial v / \partial u) \cdot (\partial u / \partial x) \quad (\text{multiply local derivatives along the path})$$

Case Study 2 — forecasting loss

Graph: $u = 2x$, $v = u + 3$, $L = v^2$, with $x = 1$.

Qty	Value
$u = 2x$	2
$v = u + 3$	5
$L = v^2$	25
$\partial L / \partial v = 2v$	10
$\partial L / \partial x = 2v \cdot 1 \cdot 2$	20

Case Study 3 — sensor calibration

$h = w_1x + b$, $\hat{y} = w_2h$, $L = \frac{1}{2}(\hat{y} - y)^2$; $x=2$, $y=5$, $w_1=1$, $b=0$, $w_2=2$.

Qty	Value
$h = 1(2)+0$	2
$\hat{y} = 2(2)$	4
$\partial L / \partial \hat{y} = \hat{y} - y$	-1
$\partial L / \partial w_2 = (\hat{y} - y) \cdot h$	-2
$\partial L / \partial w_1 = (\hat{y} - y) \cdot w_2 \cdot x$	-4

Backprop recipe: (1) forward pass to get every intermediate value; (2) start with $\partial L / \partial \text{output}$; (3) walk backward, multiplying by each local derivative; (4) for a weight, the last local factor is the input feeding that weight.

6 · Representation Power (Universal Approximation)

What a single hidden layer can do

- **Network of perceptrons:** a single hidden layer can represent **any boolean function** exactly (may need up to 2^n hidden units).
- **Network of sigmoid neurons:** a single hidden layer with **enough neurons** can **approximate any continuous function** to arbitrary accuracy — the Universal Approximation Theorem.
- Catch: "enough neurons" can be very large; depth is what makes networks efficient in practice.

Exam quick-fire (assignment-style)

- Forward pass first: hidden quantities must be computed **before** $\hat{y}$ (feedforward = inputs $\rightarrow$ hidden $\rightarrow$ output).
- Chain rule: $\partial L / \partial x$ = product of local derivatives along the path. CS2: $\partial L / \partial v = 10$, $\partial L / \partial x = 20$.
- CS3 sensor: $h=2$, $\hat{y}=4$, $\partial L / \partial \hat{y} = -1$, $\partial L / \partial w_2 = -2$, $\partial L / \partial w_1 = -4$.
- GD update: $w \leftarrow w - \eta \cdot \partial L / \partial w$. CS4: $\hat{y}=3$, $\partial L / \partial \hat{y} = -3$, $\partial L / \partial w_2 = -6$, new $w_1=1.3$, new $b=0.3$.
- Gradient = local rate of change of loss w.r.t. a parameter.
- Sigmoid: smooth, differentiable, output in $(0,1)$. One hidden layer of sigmoids $\Rightarrow$ universal approximator.